

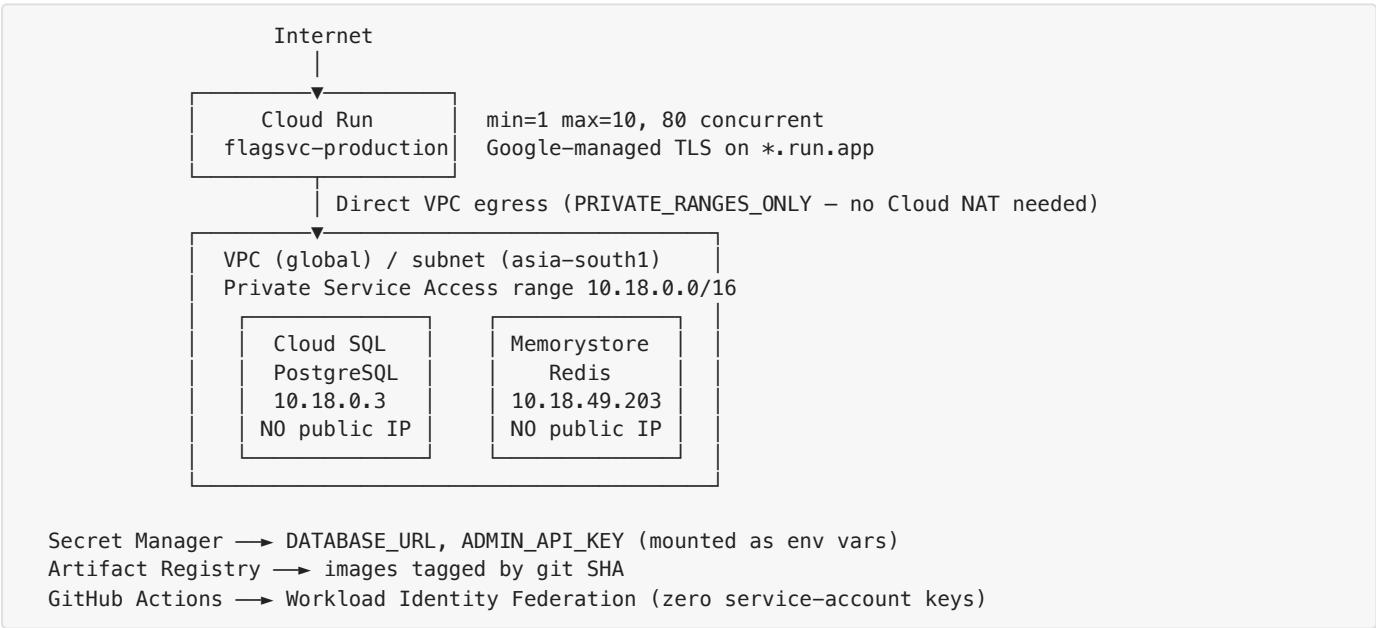
Multi-Tenant Configuration & Feature Flag Service

Backend & Platform Take-Home — Option C · Muhammad Usama · July 2026

A centralised feature flag service — a small **LaunchDarkly**. FastAPI on Cloud Run, Cloud SQL and Memorystore on a private VPC, provisioned entirely with Terraform and deployed by GitHub Actions with a 10% canary and automatic rollback. Nothing was deployed by hand.

Repository	github.com/rana-m-usama/thinking-feature-flag-service (public)
Live application	https://flagsvc-production-wqwagrz3q-el.a.run.app
Interactive API docs	/docs · readiness: /readyz
Service code	/thinking-feature-flag-service
Infrastructure	/thinking-terraform — 8 modules, 45 resources
CI/CD	Actions — passing , run #29534722144
Region / project	asia-south1 (Mumbai) / thinking-flagsvc-0e6b

1. Architecture



On a cache hit the hot path never touches **Postgres**. That is what lets a `db-f1-micro` serve the whole service: the database handles flag CRUD and audit inserts — a few writes a day — not evaluation traffic.

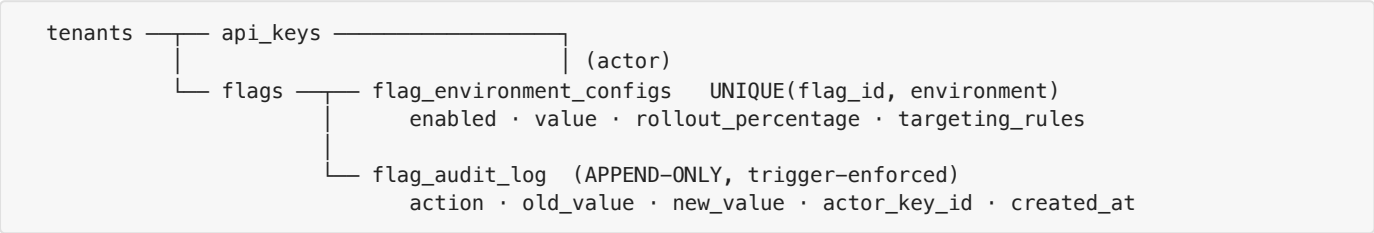
Technology choices

Choice	Why	Alternative rejected because
Cloud Run	Revision traffic-splitting <i>is</i> the canary requirement, built in. Scales to zero.	GKE : ~\$75/mo control plane plus node ops, to run one stateless container.
Direct VPC egress	No instances to pay for or patch; GA since 2024.	Serverless VPC connector : dedicated e2-micro instances, ~\$30/mo, same result.
Cloud SQL <code>db-f1-micro</code>	Hot path served from Redis; DB does occasional writes.	AlloyDB : no shared-core tier, ~\$250/mo — a database for a workload that barely has one.
Memorystore BASIC	The cache is rebuildable; every key derives from a query.	STANDARD_HA : 2× cost to replicate a component designed to be disposable.

Private IP only	No internet path to either datastore — absent from it, not firewalled off it.	Public IP + authorized networks: a firewall you can misconfigure.
------------------------	---	--

2. Data model

The decision everything hangs off: a flag's identity is separate from its per-environment state. `flags` holds what the flag *is*; `flag_environment_configs` holds what it is *doing* in dev vs staging vs production. This falls out of the brief directly — `POST /flags` takes name/type/default but no environment, while `GET /flags?environment=production` filters by one. Collapsing them forces you to either duplicate the definition three times (rename it in prod and dev drifts) or lose per-environment state entirely.



Detail	Reasoning
Two values, not one	<code>flags.default_value</code> is served when off/archived/outside the rollout; <code>configs.value</code> when on. Two is the structural minimum for the <code>string</code> type to mean anything — a flag with only a default returns it to everyone and the rollout has nothing to select <i>between</i> .
<code>tenant_id</code> denormalised	On every table, even where reachable via <code>flag_id</code> . Every composite index leads with it. This is the isolation boundary — see §5.
Configs seeded eagerly	Creating a flag writes 3 config rows, all <code>enabled=false</code> . Production is dark from the instant a flag exists, and the evaluator has no null branch on its hot path.
Rules as jsonb	Evaluation needs every rule for every flag at once — embedding them means one query, one cache entry. Cost: no SQL-level validation, hence API-boundary validation plus a <code>pg_column_size < 8192 CHECK</code> to keep the hot path off TOAST.
No <code>users</code> table	Deliberate. <code>user_id</code> is opaque and belongs to the tenant's system. Storing it makes us a data processor for every tenant's PII, doesn't scale, and is <i>wrong</i> — if we cached <code>plan:"free"</code> and the caller sends <code>plan:"enterprise"</code> , ours is stale by definition.

The audit log is immutable — enforced by the database, not by discipline

```

$ psql -c "UPDATE flag_audit_log SET new_value = '{\"tampered\":true}';"
ERROR:  flag_audit_log is append-only: UPDATE is not permitted
$ psql -c "DELETE FROM flag_audit_log;"
ERROR:  flag_audit_log is append-only: DELETE is not permitted
  
```

Its foreign keys are `RESTRICT`, not `CASCADE` — a cascading delete is still a `DELETE`, and allowing it would be a backdoor through immutability: erase the audit trail by deleting its tenant.

3. The evaluation algorithm

```
def bucket_for(flag_key: str, user_id: str) -> int:
    digest = sha256(f"{flag_key}:{user_id}".encode()).digest()
    return int.from_bytes(digest[:4], byteorder="big") % 100
```

A **pure function of two strings**. No database, no clock, no randomness. Every process on every instance computes the same bucket, forever.

Precedence — the ordering *is* the design

#	Condition	Returns
1	Archived	default_value
2	enabled = false	default_value — the kill switch outranks targeting : an operator disabling a flag during an incident must not be second-guessed by a rule someone wrote last month.
3	First matching targeting rule	that rule's value — outranks the rollout , otherwise a QA user pinned to true sits at their hash bucket and tests a coin flip.
4	bucket >= rollout_percentage	default_value
5	otherwise	value

Verified against the live GCP service

Predicted buckets: qa-bot=17, user_2c91=67, user_8f3a=82.

```
rollout  qa-bot(17)  user_2c91(67)  user_8f3a(82)
10%      false      false      false
25%      true       false      false    <- 17 enters
50%      true       false      false
70%      true       true       false    <- 67 enters
100%     true       true       true     <- 82 enters
— rollback to 25% —
25%      true       false      false    <- exactly the cohort that was added
```

Widening a rollout is purely additive. user_2c91 is at bucket 67 whether the rollout is 10% or 70% — raising the percentage only ever *adds* people. Nobody loses a feature mid-session, which is the property that makes gradual rollout usable at all. With random(), a customer would build a cart in the new flow and pay in the old one.

Why flag_key is in the hash. user_2c91 sits at bucket 67 for checkout but 6 for search — same user, opposite fates. Hashing user_id alone would put the same unlucky cohort in the first 10% of every rollout: permanent guinea pigs, and every experiment confounded by every other.

Uniformity (100k synthetic users)

```
buckets 0-9 (a 10% rollout):
10,007 users -> 10.01%
smallest bucket: 921
largest bucket: 1,055 (expect ~1,000)
```

Live string flag @50% (120 real users)

```
61 control
59 treatment

user_7 x 8 calls -> control (1 distinct)
```

Cross-language reproducibility. An independent Node implementation produces byte-identical buckets (17/75, 67/6, 82/41). SHA-256 + big-endian uint32 + mod 100 is unambiguous in every language — which is what would make client-side SDK evaluation viable. A language-native hashCode() would not be.

Modulo bias is present and ignored deliberately: $2^{*32} \% 100 == 96$, so buckets 0–95 are ~0.000002% more likely than 96–99. Correcting it costs more code than the error costs anyone. **The : separator is load-bearing** — without it ("ab", "c") and ("a", "bc") collide and one flag serves another's cohort.

4. Deployment and rollback

```
push to main (thinking-feature-flag-service/** only – terraform excluded)
↓
lint + 58 tests                ← real Postgres + Redis service containers
↓
build → push :$GIT_SHA → run migrate job (must succeed)
↓
deploy --no-traffic --tag=candidate ← boots, serves nobody
↓
smoke test candidate /readyz      ← proves it reaches Cloud SQL through the VPC
↓
10% canary → watch → 100%
↓
ANY failure → traffic back to the previous revision (health-checked first)
```

Canary, not blue-green — and there is a neat reason it is safe here. Cloud Run splits traffic *per request*, *not sticky*: the same user can hit the old revision then the new one. For most services that is a problem. Here it is fine, because evaluation is a pure function of `(flag_key, user_id)` — both revisions compute bucket 67 for `user_2c91` and return the same answer. **The determinism that makes rollouts work is what makes non-sticky canary deploys safe.**

Decision	Reasoning
Terraform owns the shape; CI owns the image and traffic split	<code>lifecycle { ignore_changes = [image, traffic] }</code> . Without it, every terraform apply reverts whatever CI deployed — an unplanned rollback triggered by an unrelated infra change — and an apply mid-canary would slam 100% of traffic onto the candidate, completing a rollout nobody approved.
Migrations as a Cloud Run Job	Before traffic shifts, never at container start where N autoscaled instances would race the same schema. <i>Consequence</i> : during a canary old and new revisions share one schema, so every migration must be backward-compatible . Add a column, deploy, backfill, drop later — never rename in one step.
Keyless CI auth	Workload Identity Federation, pinned by <code>attribute_condition</code> to this exact repository. No service-account JSON key exists anywhere — the most commonly leaked GCP credential, because it never expires and works from anywhere.
Rollback = traffic shift	Seconds, no rebuild, because the previous revision was never deleted. That is the whole reason for revision-splitting over a rolling update. CI health-checks the target first: "previous revision" does not mean "known good".

5. Testing

58 tests, run in CI on every push. 40 unit (evaluation engine) + 18 integration (isolation, environment scoping, cache separation, auth). The weight is on the engine because a bug there serves the wrong feature to real users *silently*, whereas a bug in the CRUD layer returns a 500 someone notices.

Test	Why it exists
Determinism — 1,000 calls, one result	The property the product rests on.
Exact bucket constants pinned	A deliberate change-detector. Swap the hash or drop the <code>:</code> and every live rollout silently reshuffles — users lose features mid-session. Must fail in CI, not production.
Uniformity $\pm 1\%$ over 20k users	A non-uniform hash is perfectly deterministic and passes every other test while making "10%" mean 3%.
Monotonicity	Everyone in the cohort at 25% is still in it at 70%.
Falsy values (<code>0</code> , <code>""</code>)	Legitimate flag values, not "no value". A truthiness check anywhere in the engine would serve the default and be nearly invisible.

The isolation result worth reading

The authorisation guard was **deliberately deleted** and an attacker holding a valid key asked for a victim's flags:

```
GET victim's flags with attacker key -> HTTP 200
  response body: []
  victim's real flags: ["victim.secret_flag"]

PUT victim's flag with attacker key -> HTTP 404
  victim's flag still: enabled=false
```

With authorisation entirely removed, no data leaked. The redundant `tenant_id` filter in the repository layer — filtering on the authenticated key's tenant, never the URL — held the line alone. Damage was limited to wrong status codes.

Live isolation, verified against GCP

Attack (attacker holds a <i>valid</i> key)	Result
GET / PUT / DELETE / history on another tenant	404 — not 403, which would confirm the tenant exists
Spoofed <code>tenant_id</code> in evaluate body	403 — the key is the authority, never the payload
No key / forged key	401 — not 422; a missing credential is an auth failure

6. Observability & live performance

Measured on the deployed service

```
evaluation_latency_ms:
  p50: 0.787   p95: 2.233   p99: 6.872
  samples: 451

cache: 430 hits / 21 misses
      ratio: 95.3%
```

Alerts

5xx error rate	>5% / 5min
Evaluation p95	>500ms / 5min
Uptime check	≥2 probe regions failing

Metrics derive from the structured logs, not a client library. On an autoscaled, scale-to-zero service an in-process counter is per-instance and per-lifetime: honest for one container, wrong in aggregate. The p99 you want is across the fleet, not inside a process that started 40 seconds ago.

Route templates, never resolved paths. `/api/v1/tenants/{tenant_id}/flags` , not `/api/v1/tenants/83fc.../flags` . Resolved paths embed UUIDs, so every new tenant mints a new time series forever — the monitoring bill grows with the customer list and the alert never fires because no series has enough volume. This was a real bug, caught by reading the metrics output.

The error-rate alert counts 5xx only, not 4xx. A tenant sending a bad API key produces 401s all day and that is the service *working*. The brief says "error rate" without saying whose errors; this reads it as ours.

7. Assumptions

Assumption	Why it was necessary
ADMIN_API_KEY for tenant registration	POST /tenants cannot authenticate with a tenant key — at that moment neither the tenant nor its key exists. The endpoint is unimplementable without a separate operator credential. The brief does not describe how tenant creation is authorised.
/evaluate vs /evaluate/bulk	The brief gives both the same request body and does not say how they differ. /evaluate takes an optional flag_keys to scope; omit it and they are equivalent.
environment in the PUT body	The brief's PUT mixes flag-level operations ("modify default value") with environment-level ones ("toggle on/off"), but its URL carries no environment.
Actors are API keys, not humans	The only auth mechanism in the brief is API keys — no login, no accounts, no RBAC. Real human attribution needs an identity layer not asked for.
Unknown flag_keys return {}	An SDK asking for a nonexistent flag falls back to its own default rather than getting a 500.

8. Trade-offs and known gaps

Item	Position
Load test (k6) — not done	The one requirement not met, and the single biggest gap. Everything is verified sequentially, so the connection pool under parallel load, the rate limiter, and cache stampedes on a cold key are unexercised. That is where I would expect the next real bug.
SSE / WebSocket — not done	Explicitly a bonus. It would also change the Cloud Run config: cpu_idle = true means an idle SSE connection cannot push.
No reason on the evaluate response	The engine computes one and logs it, but the brief asks only for values. It is the most useful debugging field a flag service can expose and would be my first addition.
Fixed-window rate limiting	Allows a 2× burst across a minute boundary. A sliding-log window fixes it at the cost of a sorted set per tenant. Harmless at these limits.
Connection math is deliberately tight	db-f1-micro allows ~25 connections; the pool is <i>per container</i> , so $10 \times (2+1) = 30$. Over 25 on purpose — reaching it needs all 10 instances saturated <i>and</i> holding full overflow, which for an async app whose hot path never touches the DB means a genuine incident. The fix is db-g1-small, not a smaller pool, which would move the failure inside each container.
Staging written, not applied	Own tfvars, own backend, own state prefix. Memorystore's 1GB BASIC (~\$35/mo) is the smallest SKU that exists, so two environments over the 90-day window is ~\$320 against a \$300 grant. make plan ENV=staging works; the spend is a documented decision.
Load balancer written, disabled	Google-managed certs validate by DNS against a domain you own. Without one an LB serves only plain HTTP — <i>strictly worse</i> than the free TLS already on the run.app URL. Set domain and the LB, Cloud Armor and ingress-lockdown appear (+\$18/mo).
No AWS-SCP equivalent	GCP Organization Policies require a GCP Organization, which a personal account does not have. What we get: deletion_protection on Cloud SQL (API-enforced) and prevent_destroy on the state bucket.

9. Cost — production, ~\$63/month

Cloud SQL db-f1-micro ZONAL	~\$9
Memorystore BASIC 1GB	~\$35 — the floor; no free tier, no smaller SKU
Cloud Run (min=1)	~\$5

A budget alert fires at 50/90/100% of \$300, plus a forecast-based one that lands days earlier. GCP has no hard spend cap — it is a smoke detector, not a sprinkler.

Two implicit defaults that would have been expensive, caught at apply time:

1. Cloud SQL defaults newer Postgres versions to `ENTERPRISE_PLUS`, which rejects every shared-core tier and demands `db-perf-optimized-*` — roughly **\$300+/month**, the entire credit in one month. The edition is now pinned explicitly. Had the tier happened to be compatible it would have silently created a 30× more expensive instance, and the first symptom would have been a billing alert.
2. The PSA range must be a `/16`, not a `/24`. Each service provider carves an entire *subnet* out of it: Memorystore takes a `/29`, but Cloud SQL demands a full `/24` for itself. Inside a `/24` total, Memorystore's `/29` leaves ~248 free addresses and *still* no room — failing with the thoroughly misleading "Couldn't find free blocks in allocated IP ranges". Reason about subnet blocks, not address counts.

10. Engineering notes

A few findings that only surfaced by running the thing, and which are documented in the code rather than smoothed over:

- **`/healthz` is intercepted by Cloud Run's frontend** and never reaches the container — it returns Google's own HTML 404. The proof: `/healthzz`, a path the app does not define, reaches FastAPI and returns its JSON 404. The canary probed `/healthz`, could never get a 200, and rolled back every deploy while the candidate was provably healthy. The safety mechanism was the outage. Locally it works perfectly, which is what made it invisible. Renamed to `/livez`.
- **Alembic passes the DB URL through ConfigParser**, where `%` is interpolation syntax — so any URL-encoded password (`%2B`, `%23`) is rejected by the config layer before reaching the database. Compounded by a comment claiming `#` was excluded from the password charset when it was not. A comment describing a constraint the code does not enforce is worse than no comment: it is a claim reviewers trust. The constraint now lives in the value (RFC 3986 unreserved only).
- **A rollback that made things worse.** The canary watched the *service* URL, which honours the traffic split — so 90% of probes hit the old revision. It now watches the candidate's tagged URL and health-checks the rollback target before trusting it. "Previous revision" does not mean "known good".
- **The audit log recorded no-op changes.** An idempotent `PUT` wrote `old={"rollout_percentage":25}` `new={"rollout_percentage":25}` — a log whose only job is answering "what changed" asserting a change that did not happen. Now only differing fields are recorded.